



# Java

# Características

- ▶ **Orientada a objetos:** tudo são classes ou instâncias de classes (com exceção dos tipos primitivos);
- ▶ **Sem ponteiros:** não permite a manipulação direta de endereço de memória;
- ▶ **Coletor de lixo:** mecanismo automático de gerenciamento de memória;
- ▶ **Permite *multithreading*;**

# Características

- ▶ **Independente de plataforma:** programas java são compilados para uma forma intermediária (bytecodes);
- ▶ **Máquina Virtual Java (JVM):** responsável por interpretar os bytecodes;
- ▶ **Tratamento de exceções.**

# Processo de Compilação e Interpretação

- ▶ **Execução:** a JVM interpreta o programa em tempo de execução;
- ▶ **Tempo de execução:** período em que um programa permanece em execução;
- ▶ **Tempo de compilação:** período em que o código é compilado para gerar um programa.

# Processo de Compilação e Interpretação

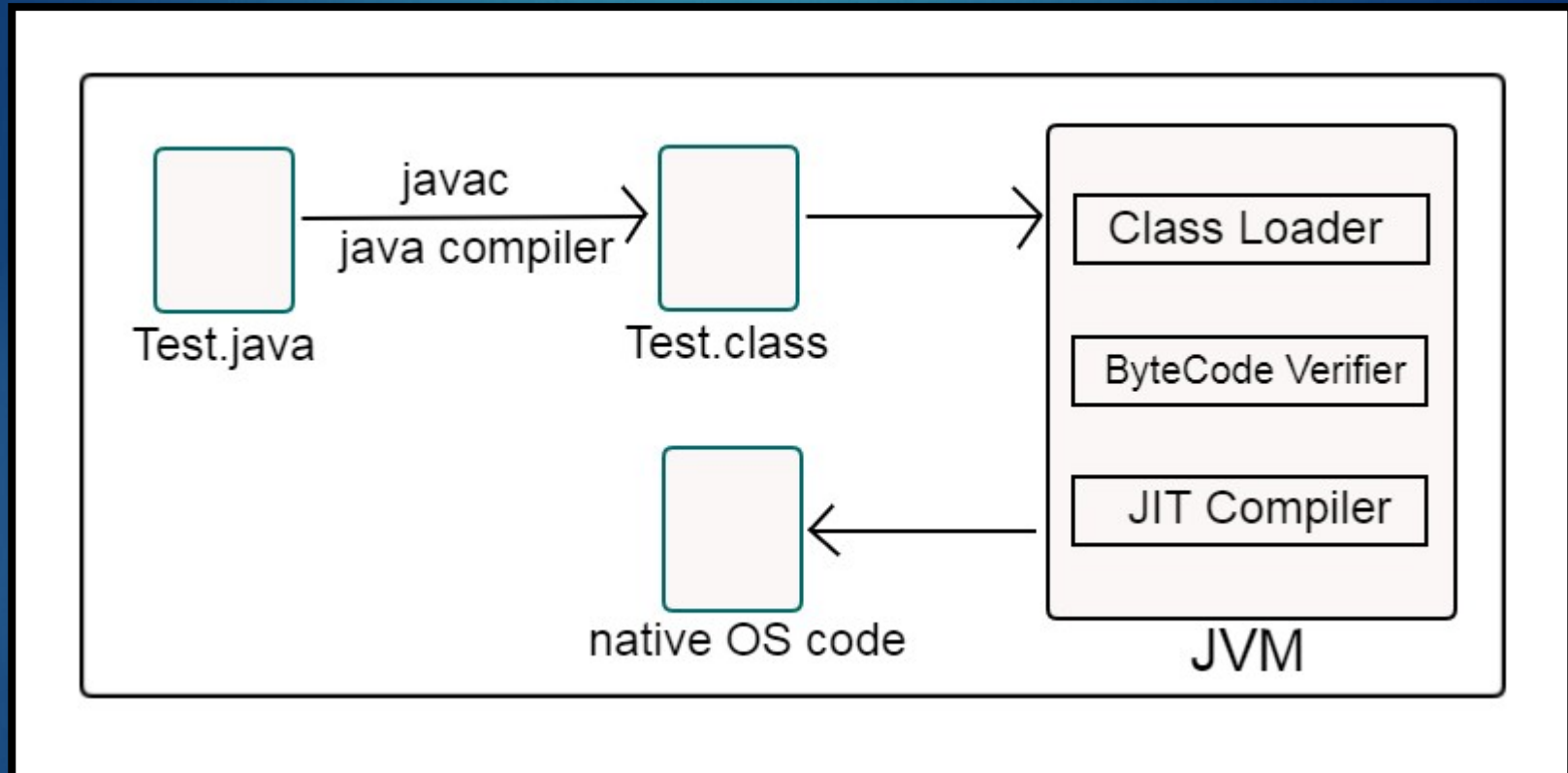


Imagem retirada de: <https://programdiary.com>

# Sintaxe

- ▶ Todo programa deve ter um nome de classe igual ao nome do arquivo;
- ▶ O método **main()** é obrigatório;

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

# Comentários

- ▶ `// comentário` - comenta uma única linha;
- ▶ `/* comentário */` - comenta múltiplas linhas.

```
/* O código abaixo vai imprimir as  
   palavras Hello World na tela.*/  
System.out.println("Hello World");
```



# Variáveis

- ▶ Uma variável é um espaço em memória para armazenar valores;

- ▶ Tipos primitivos:

| Data Type | Size    |
|-----------|---------|
| byte      | 1 byte  |
| short     | 2 bytes |
| int       | 4 bytes |
| long      | 8 bytes |
| float     | 4 bytes |
| double    | 8 bytes |
| boolean   | 1 bit   |
| char      | 2 bytes |

Imagem retirada de:

<https://www.w3schools.com/>



# Variáveis – Tipos não primitivos

- ▶ São chamados de **tipos de referência** porque se referem a objetos;
- ▶ São definidos pelo programador (Exceção: String);
- ▶ Tipos:
  - ▶ **String** – armazena texto;
  - ▶ **Arrays** – estruturas que podem conter múltiplos elementos;
  - ▶ **Classes.**

# Variáveis

## ► Exemplos de declarações:

```
int myNum = 5;  
float myFloatNum = 5.99f;  
char myLetter = 'D';  
boolean myBool = true;  
String myText = "Hello";
```

# Variáveis

- Declarando muitas variáveis:

```
int x = 5;  
int y = 6;  
int z = 50;  
System.out.println(x + y + z);
```

- O código acima pode ser simplificado:

```
int x = 5, y = 6, z = 50;  
System.out.println(x + y + z);
```

# Variáveis

- Um valor para múltiplas variáveis:

```
int x, y, z;  
x = y = z = 50;  
System.out.println(x + y + z);
```

# Operadores

- ▶ Utilizados para realizar operações com valores e variáveis;
- ▶ Tipos:
  - Aritméticos;
  - De Atribuição;
  - De Comparação;
  - Lógicos;
  - Bitwise (Bit-a-Bit).

# Operadores - Aritméticos

| Operator | Name           | Description                            | Example          |
|----------|----------------|----------------------------------------|------------------|
| +        | Addition       | Adds together two values               | $x + y$          |
| -        | Subtraction    | Subtracts one value from another       | $x - y$          |
| *        | Multiplication | Multiplies two values                  | $x * y$          |
| /        | Division       | Divides one value by another           | $x / y$          |
| %        | Modulus        | Returns the division remainder         | $x \% y$         |
| ++       | Increment      | Increases the value of a variable by 1 | <code>++x</code> |
| --       | Decrement      | Decreases the value of a variable by 1 | <code>--x</code> |

Imagem retirada de: <https://www.w3schools.com/>

# Operadores – De atribuição

| Operator | Example | Same As    |
|----------|---------|------------|
| =        | x = 5   | x = 5      |
| +=       | x += 3  | x = x + 3  |
| -=       | x -= 3  | x = x - 3  |
| *=       | x *= 3  | x = x * 3  |
| /=       | x /= 3  | x = x / 3  |
| %=       | x %= 3  | x = x % 3  |
| &=       | x &= 3  | x = x & 3  |
| =        | x  = 3  | x = x   3  |
| ^=       | x ^= 3  | x = x ^ 3  |
| >>=      | x >>= 3 | x = x >> 3 |
| <<=      | x <<= 3 | x = x << 3 |

Imagem retirada de: <https://www.w3schools.com/>



# Operadores – De comparação

| Operator | Name                     | Example |
|----------|--------------------------|---------|
| ==       | Equal to                 | x == y  |
| !=       | Not equal                | x != y  |
| >        | Greater than             | x > y   |
| <        | Less than                | x < y   |
| >=       | Greater than or equal to | x >= y  |
| <=       | Less than or equal to    | x <= y  |

Imagem retirada de: <https://www.w3schools.com/>

# Operadores – De lógicos

| Operator | Name        | Description                                             | Example                                       |
|----------|-------------|---------------------------------------------------------|-----------------------------------------------|
| &&       | Logical and | Returns true if both statements are true                | <code>x &lt; 5 &amp;&amp; x &lt; 10</code>    |
|          | Logical or  | Returns true if one of the statements is true           | <code>x &lt; 5    x &lt; 4</code>             |
| !        | Logical not | Reverse the result, returns false if the result is true | <code>!(x &lt; 5 &amp;&amp; x &lt; 10)</code> |

Imagem retirada de: <https://www.w3schools.com/>

# Condições

- ▶ Declarações condicionais realizam ações de acordo com condições específicas.
- ▶ Tipos:
  - **if** – executa o código se a condição for verdadeira;
  - **else** – executa o código se a condição especificada no if for falsa;
  - **else if** – especifica uma nova condição de teste, se a primeira condição for falsa.

# Condições

```
int time = 22;  
if (time < 12) {  
    System.out.println("Bom dia!");  
} else if (time < 18) {  
    System.out.println("Boa tarde!");  
} else {  
    System.out.println("Boa noite!");  
}
```

# Condições – Operador ternário

- ▶ Abreviação do if else;
- ▶ Pode substituir múltiplas linhas de código;

```
int tempo = 20;  
String resultado;  
resultado = (tempo < 18) ? "Bom dia." : "Boa noite.";  
System.out.println(resultado);
```

# Condições – Switch

- Substitui várias declarações if else.

```
int day = 4;
switch (day) {
    case 6:
        System.out.println("Today is Saturday");
        break;
    case 7:
        System.out.println("Today is Sunday");
        break;
    default:
        System.out.println("Looking forward to the Weekend");
}
// Outputs "Looking forward to the Weekend"
```

Imagem retirada de: <https://www.w3schools.com/>

# Laços de Repetição - While

- Permitem executar um bloco de código até que uma condição seja alcançada.

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```



# Laços de Repetição – Do/While

- ▶ Executa o código ao menos uma vez.

```
int i = 0;  
do {  
    System.out.println(i);  
    i++;  
}  
while (i < 5);
```

# Laços de Repetição – For

- Para situações onde se sabe a quantidade de vezes que um bloco de código será executado.

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

# Break

- Encerra a execução de um laço ou condição.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        break;  
    }  
    System.out.println(i);  
}
```

# Continue

- Pula uma iteração no laço.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    System.out.println(i);  
}
```

# Array

- Armazena múltiplos valores de determinado tipo.

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
for (int i = 0; i < cars.length; i++) {
    System.out.println(cars[i]);
}
```